Bah Humbug Allocator

A Proportional Fairness Algorithm for the Optimized Allocation of Operational Modules to a Pool of Operators

P. Riva

Abstract

This report describes the design and mathematical formalization of an algorithm for the optimized allocation of operational modules to a pool of operators, based on their previously provided individual availabilities. The proposed system is built on two complementary mechanisms: a dynamic weighting that values the coverage of periods with higher supply scarcity, and a proportional fairness system. Each module is assigned a weight w_i, calculated through a logarithmic function of the excess supply relative to the demand. The adopted loss function minimizes the discrepancy from a proportional target $T_j(\alpha)$, which represents the fair share of the workload assigned to each operator relative to the actual demand of the system. A hyperparameter $\alpha \in [0, 1]$ allows calibrating the balance between an egalitarian distribution and a meritocratic distribution based on the flexibility offered. The final allocation is obtained by minimizing this quadratic loss function subject to constraints of demand satisfaction, availability compliance, participation guarantee, and the binary domain of the decision variables. The resulting problem is a Mixed-Integer Quadratic Program (MIQP), whose structure is analytically described in canonical form.

1 Introduction

This project stems from the need for an allocation of operational modules to a pool of operators that accounts not only for the availability of the operators, but also for its scarcity, and that compensates the effort of those who cover the most "difficult" periods. Offering availability when few others do is worth more to the system than offering it when many are available, and the allocation should recognize this difference rather than counting every shift the same.

The application domain examined in this work is characterized by some very specific constrains:

1. The operational modules to be allocated and the related personnel requirements are strictly predetermined;
2. Operator availabilities are collected in advance;
3. The organizational system is design so that the aggregate availability provided by the operators systematically exceeds the actual demand, creating a condition of global supply overabundance.

In this setting, not every operational slot is worth the same. Distributing assignments in a circular or random manner, iterating over availabilities until the demand is saturated, treats every slot as identical and ignores a critical factor: the relative scarcity of the supply. Although the system is globally overabundant, the distribution of availabilities presents severe local bottlenecks.

A practical example is the assignment of technical or logistical consulting shifts in a given month. "Module A" (e.g., a central shift on a weekday during low season) requires 5 operators but gathers 20 availabilities. "Module B" (e.g., an evening shift on a holiday eve or during peak season) still requires 5 operators, but gathers exactly 5 availabilities. An allocation that counts every slot equally will randomly distribute Module A, and will be forced to assign Module B to the only 5 available operators. The operational result is dysfunctional: the operator who "saved" the system by covering the inconvenient Module B will have that shift counted exactly the same as the one who covered the convenient Module A. This paradigm not only disincentivizes future flexibility but also ignores the opportunity cost of the operators, who will find no advantages in offering availability in those periods if they are not rewarded for their effort.

To correct this, the algorithm proposed in this work tackles the problem on two distinct mathematical levels. At the module evaluation level, it introduces a dynamic weighting system that increases with the excess supply: returning to the example, Module B will weigh zero in the calculation of the operator's workload, allowing them to receive further reward assignments, while Module A will carry the maximum weight.

At the objective function level, the algorithm rejects the naive approach of attempting total saturation of individual availabilities which is mathematically impossible, as supply exceeds demand. Instead, it adopts a principle of proportional fairness: the assignment target is not the operator's absolute availability, but a quota calculated dynamically based on the flexibility demonstrated relative to the entire pool. This approach is governed by a trade-off parameter $\alpha \in [0, 1]$, which allows the decision-maker to mathematically balance baseline equity and the meritocratic recognition of flexibility.

2 Definition of the Module Score (w_i)

The core of the algorithm lies in the function used to calculate the score (or weight) of each task. This system must guarantee three fundamental properties:

1. **Normalization:** The value must fall within the interval $[0, 1]$;
2. **Inverse proportionality to scarcity:** The smaller the excess supply, the lower the weight must be;
3. **Sensitivity to distribution:** It must distinguish not only between "abundance" and "scarcity," but also evaluate how critical an assignment is compared to the average of the others.

2.1 Mathematical Formalization

The total number of modules is denoted by n and the number of operators who have provided at least one availability by k. The following vectors are defined:

- **Total Availability Vector (S):** An $n \times 1$ vector where each element s_i represents the sum of the availabilities received for the i-th module.

$$s_i = \sum_{j=1}^{k} s_{ij} \quad \text{with } s_{ij} \in \{0, 1\}$$

- **Demand Vector (D):** An $n \times 1$ vector where each element d_i indicates the number of operators required for the proper execution of module i.

The weight w_i assigned to the i-th module is calculated using the following function:

$$w_i = \max \left\{ 0, \frac{\log(s_i - d_i + \delta)}{\max_i \left[\log(s_i - d_i + \delta)\right]} \right\}$$

where $\delta > 0$ is a small offset that avoids the $0/0$ situation.

The use of this specific logarithmic formulation analytically meets the initial requirements:

- **Maximum Value ($w_i = 1$):** Achieved in modules where the excess supply $(s_i - d_i)$ is maximal. These are "easy" to cover and carry full points in the operator's assignment count.
- **Minimum Value ($w_i = 0$):** Achieved when the excess supply is zero ($s_i = d_i$). In this scenario of maximum scarcity, the assignment carries "no weight," burdening the assignment count minimally.

3 The Allocation Algorithm

Once the module weights are defined, the focus shifts to the actual optimization phase. The problem consists of distributing the workload among the operators fairly based on the number and distribution of the availabilities provided.

3.1 Decision Variables and Complexity Reduction

The number of modules is denoted by n and the number of operators who have provided at least one availability by k. The following matrices are introduced:

- **Availability Matrix ($S \in \{0, 1\}^{n \times k}$):** Where $s_{ij} = 1$ if operator j is available for module i, subject to the constraint $\sum_{j=1}^{k} s_{ij} > 0 \quad \forall_{i=1,\dots,n}$:

$$\boldsymbol{S}_{n \times k} = \begin{bmatrix} s_{11} & s_{12} & \dots & s_{1k} \\ s_{21} & s_{22} & \dots & s_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ s_{n1} & s_{n2} & \dots & s_{nk} \end{bmatrix}$$

- **Assignment Matrix ($A \in \{0, 1\}^{n \times k}$):** Represents the decision variables of the model:

$$\boldsymbol{A}_{n \times k} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1k} \\ a_{21} & a_{22} & \dots & a_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nk} \end{bmatrix}$$

Analytically, the problem would require the estimation of $n \times k$ parameters. However, the nature of the problem allows for a reduction of the search space: a task can only be assigned if there is a prior availability ($x_{ij} = 1 \implies s_{ij} = 1$).

The matrix of feasible assignments can therefore be defined using the Hadamard product operator:

$$\boldsymbol{X}_{n \times k} = \boldsymbol{A}_{n \times k} \odot \boldsymbol{S}_{n \times k}$$

This approach drastically reduces the number of actual variables in the solver, limiting them solely to the non-zero entries of matrix S.

3.2 Naive Loss Function

Before introducing the adopted formulation, it is necessary to analyze why a naive approach, directly minimizing the discrepancy between offered availability and received assignments, produces systematically distorted results. Understanding these issues is crucial to motivating the design choice of the correct loss function.

The structural paradox of scarcity. The total availability offered by operator j is denoted by $S_j = \sum_{i=1}^{n} s_{ij}$, and the total demand of the system by $D_{tot} = \sum_{i=1}^{n} d_i$. In a network with many operators, the aggregate availability systematically exceeds the demand:

$$S_{tot} = \sum_{j=1}^{k} S_j \gg D_{tot}$$

A naive approach that minimizes $\sum_{j=1}^{k} (S_j - \sum_i w_i x_{ij})^2$ implicitly asks the system to assign each operator exactly as many modules as they offered. This is **mathematically impossible**: the total demand is not sufficient to satisfy all expectations simultaneously. The solver, forced to operate in an overdetermined system, develops two compensatory problems:

1. **The "Elephant" effect.** The quadratic penalty disproportionately amplifies large deviations. Guided by the quadratic structure, the solver concentrates assignments on high-availability nodes to minimize the overall penalty, preferentially assigning them tasks with weight $w_i \approx 1$. As a result, operators with greater availability caputre a disproportionate share of the available demand.

2. **The "Starvation" of the middle class.** To compensate for the over-assignment to highly available individuals, the solver pushes operators with average availability toward the lower bounds. Resources that offered moderate flexibility are crushed to the minimum guaranteed by the constraints, mainly 1 assignment, regardless of their actual relative utility. This phenomenon contradicts the proportional objective of the system.

3.3 The Proportional Fairness Loss Function

The solution to the structural paradox consists of replacing the absolute target S_j with an ideal proportional target T_j, which represents the fair share of demand owed to operator j as a function of their relative flexibility compared to the network.

Definition of the ideal target T_j. The following aggregate quantities are introduced:

- **Total Demand:** $D_{tot} = \sum_{i=1}^{n} d_i$
- **Availability of operator j:** $S_j = \sum_{i=1}^{n} s_{ij}$
- **Total System Availability:** $S_{tot} = \sum_{j=1}^{k} S_j$

The ideal target for operator j is the portion of total demand proportional to their relative contribution to the system:

$$T_j = D_{tot} \cdot \frac{S_j}{S_{tot}}$$

For instance, if the system demands $D_{tot} = 100$ total allocations and operator j offers 10% of all collected availabilities, their fair target is $T_j = 10$ assignments. By the linearity of the sum, $\sum_{j=1}^{k} T_j = D_{tot}$, which ensures that the targets distribute exactly the total demand among the resources, measured in number of assignments.

The social compromise parameter α. The purely proportional formulation exclusively rewards the flexibility offered, without guaranteeing a fair minimum baseline independent of availability. To control this aspect, the hyperparameter $\alpha \in [0, 1]$ is introduced, which interpolates between a flat egalitarian distribution and a proportional meritocratic distribution:

$$T_j(\alpha) = (1-\alpha) \cdot \underbrace{\frac{D_{tot}}{k}}_{\text{flat quota}} + \alpha \cdot \underbrace{D_{tot} \cdot \frac{S_j}{S_{tot}}}_{\text{proportional quota}}$$

The edge cases have a direct interpretation:

- $\alpha = 0$ **(pure egalitarianism):** the system ignores differential availabilities and attempts to distribute the work equally among all operators. Availability acts only as an admissibility constraint.
- $\alpha = 1$ **(pure proportionality):** the target for each resource is proportional to its share of availability within the system. Those who offer more slots receive more assignments.
- $\alpha \in (0, 1)$**:** a compromise is achieved between guaranteeing a uniform baseline and recognizing greater flexibility. A value of $\alpha = 0.7$ balances basic equity and an incentive for availability.

In all cases $\sum_{j=1}^{k} T_j(\alpha) = D_{tot}$, a property that keeps the system globally consistent with total demand.

The Loss Function. Substituting the absolute target S_j with the proportional target $T_j(\alpha)$, the loss function becomes:

$$\ell_\alpha(X) = \sum_{j=1}^{k} \left(T_j(\alpha) - \sum_{i=1}^{n} w_i x_{ij} \right)^2$$

In matrix form:

$$\ell_\alpha(\boldsymbol{X}) = \left\| \boldsymbol{T}(\alpha) - (\boldsymbol{W}\boldsymbol{X})^T \right\|^2$$

where $\boldsymbol{T}(\alpha) \in \mathbb{R}^k$ is the vector of individual targets.

Why does this formulation resolve the pathologies?

- **Elimination of the Elephant effect:** an operator with very high availability (e.g., 10% of the total) will have a target proportioned to that 10%, rather than an unreachable nominal value. The squared error is drastically reduced, lowering the mathematical pressure and the solver's incentive to over-assign.
- **Recovery of the middle class:** by mitigating the impact of top offerers, available assignments are freed up for medium-availability resources, which can now receive an adequate number of tasks instead of being crushed to the minimum guaranteed constraint.
- **Compatibility with the w_i weight system:** the weighting mechanism for critical modules remains intact. If a resource covers inconvenient shifts, the sum $\sum_i w_i x_{ij}$ grows slowly, allowing it to accumulate more actual allocations before reaching $T_j(\alpha)$, consistent with the system design.

4 Canonical Formulation

The optimization problem can be summarized as follows. Given the proportional targets $T_j(\alpha)$ defined in Section 3.3, the problem is:

$$\min_X \sum_{j=1}^{k} \left(T_j(\alpha) - \sum_{i=1}^{n} w_i x_{ij} \right)^2$$

where

$$T_j(\alpha) = (1-\alpha)\frac{D_{tot}}{k} + \alpha \cdot D_{tot} \cdot \frac{S_j}{S_{tot}}$$

Subject to the following constraints:

1. **Demand Satisfaction:** Each task i must have exactly the required number of operators d_i:
$$\sum_{j=1}^{k} x_{ij} = d_i, \quad \forall i \in \{1, \dots, n\}$$
2. **Availability Constraint:** A task can be assigned only if the operator is previously available:
$$x_{ij} \leq s_{ij}, \quad \forall i, j$$
3. **Participation Guarantee:** Every operator who provided at least one availability must receive at least one assignment:
$$\sum_{i=1}^{n} x_{ij} \geq 1, \quad \forall j \in \{1, \dots, k\}$$
4. **Binary Domain:** Assignment variables must be boolean:
$$x_{ij} \in \{0, 1\}, \quad \forall i, j$$

5 Implementation and Numerical Resolution

To translate the analytical model into executable code, CVXPY [1] was adopted. The use of CVXPY offers two critical advantages for the allocation system:

- **Symbolic Abstraction:** It allows defining the loss function $\ell(X)$ and the participation and demand constraints in a nearly literal manner compared to the mathematical form.
- **Search Space Reduction:** Through the use of the Hadamard operator, the system instantiates decision variables only for the (module, operator) pairs where a prior availability exists ($s_{ij} = 1$), drastically reducing the dimensionality of the assignment matrix and accelerating solver convergence.

The computational engine selected for resolution is **SCIP** (Solving Constraint Integer Programs) [2, 3], integrated via the CVXPY interface. The choice of SCIP is motivated by the following technical features:

- **Nonlinearity Handling**: Unlike standard linear solvers, SCIP is capable of handling quadratic objectives within a *Branch-and-Cut* framework.
- **Combinatorial Efficiency**: It uses advanced presolving and constraint propagation techniques that are essential for handling the "Participation Guarantee" constraints, which dictate that every row (or column, depending on structure) of the assignment matrix meets minimum thresholds.

The search is stopped when the relative optimality gap falls below 0.1% or when the time limit (3600 s by default) is reached.

References

[1] Diamond, S., & Boyd, S. (2016). CVXPY: A Python-embedded modeling language for convex optimization. Journal of Machine Learning Research, 17(83), 1-5.

[2] Achterberg, T. (2009). SCIP: solving constraint integer programs. Mathematical Programming Computation, 1(1), 1-41.

[3] Bolusani, S., et al. (2024). The SCIP Optimization Suite 9.0. arXiv preprint arXiv:2402.17702.